PREDATOR RF

Operator Guide

Pick this up cold. Zero to a working RF picture in under 30 minutes.

The single document. If the prior operator is unavailable, this is everything you need to pick the system up cold and operate with it.

Rev 2 — June 2026 · Sensor SA beacon + CoT type corrections

Table of Contents

1. What this is, in one paragraph

Predator RF is a phone-first software-defined-radio cockpit for a solo SIGINT operator. You plug a USB SDR (a small dongle that acts as a radio receiver) into an Android phone or a Linux box, point an antenna at the world, and the app shows you what's transmitting in real time: frequency, signal power, GPS-stamped location, and whether you've seen it before. It records baselines so the next sweep flags only what's *new* in the area. It automates band scans with target and exclude lists, links to other phones or Raspberry Pi sensors as a private fleet (called Kujhad), geolocates transmitters by time-difference-of-arrival (TDOA) when you have two or more GPS-synchronized nodes, decodes P25 / RTL433 / POCSAG / ADS-B / AIS, and pushes selected detections to ATAK / TAK as Cursor-on-Target (CoT) alerts. The whole system is **receive-only** — there is no transmit path anywhere. Every output (CoT export, AutoTasker re-tunes) requires the operator to explicitly arm it. Built on the SDR++ core. Sideloads as an APK; the Python backend ships as a systemd service.

2. Two deployment paths (you can run one or both)

Path 1 — Phone (Android)

The C++ Predator RF app on a Galaxy S22 (or any Android 10+ device), driving a USB-C SDR through an OTG cable. Standalone — no other infrastructure needed. The phone IS the cockpit. Optionally peers with other phones or RPi sensors via Kujhad over Wi-Fi or a private overlay (ZeroTier / Tailscale).

Path 2 — Linux TOC + RPi sensor nodes (Python backend)

A Python backend service (predator-rf.service) runs on a Linux operator workstation or Raspberry Pi. It:

- Owns persistence (SQLite mission ledger — a local database that logs everything).
- Aggregates events from one or more Kujhad-equipped sensor nodes (phones, RPi-with-SDR, etc.) via the Kujhad HTTP API.
- Runs the TDOA coordinator, decision engine, AutoTasker, and CoT emitter centrally.
- Exposes a REST + SSE API on port 8000 (token-protected) for dashboards or chaining into a higher TOC (CoC mode).

Tip: The two paths mix freely. A typical mission: one Linux workstation running the backend, three phones in the field sharing their picture into Kujhad, one or two RPi sensors at fixed points for unattended overwatch. All fuses into one operator screen.

3. Minimum kit (you need ALL of these)

Item	Why you need it
Android phone, Android 10 (API 28) or newer	Runs the app
USB-C OTG cable or adapter (USB-C → USB-A female)	Connects the SDR to the phone
USB SDR — RTL-SDR Blog v4 ($40) minimum; HackRF One ($340) is the upgrade	The actual radio hardware. Without this, there is nothing to receive.
An antenna — even the rubber duck shipped with the SDR works to start	Without one you receive nothing
The Predator RF APK	Built from the GitHub repo on Windows, or supplied by your team

No internet required after install. No subscription. No account.

A full tiered bill of materials (cheap → full fleet) is in § 24.

4. First-time install (sideload)

Predator RF is not on the Play Store. You install it manually by 'sideloading' — here's exactly how:

1. **Unlock developer options:** Settings → About phone → tap *Build number* seven times until it says 'Developer mode is ON'.
2. Settings → Developer options → turn on *Install via USB* and *USB debugging*.

3. Get the APK file onto the phone (USB cable from a computer, Google Drive, email — anything that puts the file in your Downloads folder).

4. On the phone: open the Files app → Downloads → tap app-debug.apk.

5. When Android asks 'Install unknown apps?' → Allow for whatever app you used to open it → Install.

6. Open Predator RF.

When the app first launches it will ask for these permissions — **grant all of them:**

• **Location ('While using the app')** — required for GPS-stamped hits and TDOA.

• **Storage / Files** — required for saving baselines and exporting CSVs.

• **USB device access** — pops up the first time you plug in your SDR. Tap *Always allow for this device*.

If you skip any permission: the app still runs, but the matching feature goes completely silent — no map fix, no exports, no SDR.

---

5. Plug in the radio and start receiving

1. Connect your SDR to the phone with the USB-C OTG adapter. Antenna goes on the SDR.

2. The first time you plug in, a system dialog pops: 'Open Predator RF when this USB device is connected?' — check the box and tap OK.

3. In the app, tap the **source dropdown** (top-left, says None until a radio is selected) → pick the type matching what you plugged in (RTL-SDR, Airspy, HackRF, etc.).

4. Tap ■ **Start Listening** (big button under the source dropdown).

5. The waterfall (the scrolling coloured display) starts moving. You're live.

Waterfall stays black? See § 23 Troubleshooting.

---

6. The eight tabs (left-side rail, top to bottom)

Everything is organized into eight tabs labeled by their code on the left rail.

Code	Name	What it's for
SPEC	Spectrum	Live waterfall + tuner. Where you actually look at signals.
HITS	Hits & Events	Every signal the app has noticed, plus the running event log.
NET	Network	Catalog of known networks / talkgroups / aliases — your rolodex of emitters.
MAP	Map	GPS-stamped hits on a map, with TDOA fix dots, error ellipses, and node positions.
MIS	Mission Config	Mission mode (Manual / Classify / Scan / QuickScan), search bands, targets, excludes, dwell timing.
KUJ	Kujhad Fleet	Link this phone to other Predator RF phones or Raspberry Pi sensors over a network.
SYS	System	App settings — modules, themes, decoder bridges, ATAK/CoT, baseline comparison, TLS fingerprint.
BASE	Baseline	Record normal noise floor, save it, suppress it so only NEW signals fire as hits.

---

7. Mission modes (set on the MIS tab)

Mode	What it does	When to pick it
Manual	Direct operator tuning and marker ownership. Nothing automated.	Free recon, training, demonstration.
Classify	You stay in manual control while idle resources watch the band — passively classifying whatever crosses the threshold.	When you're working a known signal but want background awareness.
Scan	Automated search-and-target workflow across all configured bands with full target/exclude logic.	The default field mode for a real operation.
QuickScan	Rapid single-marker sweep for a quick area check — no target persistence.	Walk into a new area, spend 60 s seeing what's hot, decide whether to stay.

In a fleet: the operator workstation typically runs Scan while remote sensors run Scan with their own band lists tuned to their physical position.

---

8. The five day-one workflows

8.1 Just look around (passive recon)

1. SPEC tab → set the source → ■ Start Listening.
2. Drag the waterfall left/right to retune. Pinch to zoom the view bandwidth.
3. Tap a peak to drop a marker on it.
4. To name it: tap the marker → Assign Marker → tap 'Tap to edit' in the popup → type a label.

8.2 Mark a hit and send it to your team

1. With a marker on a signal, tap *Assign Marker* if it isn't already named.
2. The marker becomes a hit — visible on the HITS tab.
3. To push to ATAK / TAK: SYS tab → ATAK / CoT section → enter your TAK server IP + port → Enable CoT. The hit appears on the ATAK map as a sensor marker at your phone's current GPS location, with a chat message showing the frequency and power.

8.3 Record a baseline so you only see NEW signals

The first time you set up in a new area, run a baseline so the app learns what's 'normally here' (paging towers, nearby commercial radios, broadcast leakage). Then any signal NOT in the baseline is flagged as new.

1. BASE tab → Frequency Ranges → set Range Name, Start Hz, Stop Hz → **+ Add Range**. Repeat for each band. (Tap 'From Current View' to grab the current waterfall band.)
2. Recording section → set a filename (or leave blank for auto-naming) → tap ■ **START RECORDING** (green).
3. Let it run for at least 5 minutes (longer = better — 30 min for a thorough sweep).
4. ■ STOP RECORDING → Save to File.

5. SYS tab → Baseline Comparison → load the file → enable Scan against baseline. Set a threshold (default 6 dB). Now only signals exceeding the baseline by that margin become hits.

8.4 Run an automated scan across multiple bands

1. MIS tab → Mission Mode → **Scan**.
2. Search Bands section → add the bands you want swept (Name + Start Hz + Stop Hz) → **+ Add Band**.
3. Targets section — frequencies you specifically want to flag as priority (optional).
4. Excludes section — frequencies to ignore (your own gear, known broadcast carriers).
5. Dwell + QuickScan Delay + Duration — defaults are sane. Increase dwell for weak intermittent signals.
6. Back to SPEC → ■ Start Listening. The app sweeps the bands and drops hits on the HITS tab as it finds them.

8.5 Link a second phone or RPi sensor

See § 11 for the full walkthrough. Two-minute version:

- **On the device to publish from:** KUJ → Listen section → set port + Device name + API key → toggle Listen ON.
- **On the operator phone:** KUJ → Add Peer → enter Name, Host (IP), Port, API key. Toggle Mirror peer spectrum to view their waterfall.

---

9. Editing fields on a touchscreen

Every editable field works the same way:

1. Tap the field. A popup opens at the *top* of the screen, above the keyboard.
2. The keyboard slides up automatically.
3. Type. The popup is pinned high so the keyboard never covers it.
4. Tap **OK** (or hit Enter) to commit. Cancel to discard.

Letters not appearing? Some keyboards (composing / autocorrect-heavy keyboards) don't work inside the NativeActivity engine. If a popup goes blank when you type letters, switch your keyboard to a non-composing one (e.g. Google Keyboard with autocorrect off) for the duration of the edit. Numeric input always works regardless of keyboard.

---

10. The status bar at the top

Indicator	Colour	What it means
Thermal	Green / dim grey	Phone temperature is fine
	Orange	SoC heating up; expect frame rate drops
	Red	Severe throttling — lower sample rate, get out of direct sun
GPS	Green	Lock acquired, position fresh (age < 60 s)
	Yellow	Lock but ageing — TDOA will refuse this node soon
	Red / dim	No lock — TDOA disabled, map uses last known position
Kujhad	Green	All peers connected

	Yellow	Some peers down
	Red	None reachable
CoT	Off / dim	CoT export disabled
	Green	Enabled, packets going out
	Yellow	Enabled but holding at the manual-approval queue

---

11. Kujhad fleet — protocol, pairing, TLS pinning

11.1 What it is

Kujhad is the private peer protocol that links Predator RF instances together. Each instance can be a **Device** (publishes its RF state and event stream) and/or a **Controller** (connects to Devices and mirrors their state). Hub-and-spoke — you pick which devices to see. Think of it as a secure private radio network for your sensor nodes.

11.2 The wire protocol

Simple HTTP/1.1 + JSON, with a single API key in the X-Kujhad-Key header on every request. Default port **41947**.

> **Note:** The Python backend's Kujhad client defaults to port 5259 in the env-var schema — always set the actual port the Device is listening on in your FLEET_NODES config.

Endpoint	Purpose
GET /v1/identify	Device name, version, role, hardware profile
GET /v1/gps	Current GPS fix
GET /v1/state	Mission mode, scan status, threshold, search bands, decoder roster
GET /v1/events?since=N	Hit / decoded-event stream since serial N
GET /v1/timing	Clock source, PPS lock, offset, drift, last-sync age
POST /v1/command	Issue a typed command: tune, scan, mission, identify

> **Hard safety boundary:** Any command in the tx class is rejected unconditionally by the dispatcher. The whole module never opens a transmit path.

11.3 Make this device discoverable (publish)

1. KUJ tab → Listen section.
2. Listen port — leave at default (41947) unless you have a port conflict.
3. Device name — short identifier (e.g. alpha-truck, bravo-roof).
4. API key — leave the auto-generated 32-hex-char key, or paste your own. Every Controller pairing to this Device needs this same key.
5. Advertise address — usually leave blank; the app picks the best interface.
6. Toggle **Listen ON**.

The KUJ tab shows a green 'Listening on <addr>:<port>' banner. Hand the address + port + API key to whoever needs to pair.

11.4 Pair to a peer (subscribe)

1. KUJ tab → Add Peer.
2. Name — anything memorable.
3. Host — the publisher's IP address.
4. Port — the publisher's listen port.
5. API key — must match the publisher exactly, character-for-character.
6. TLS — leave OFF unless you've armed TLS on the publisher (see § 11.5).
7. Tap **Add peer**.

The peer appears in the list with a status dot. Toggle *Mirror peer spectrum* to view that peer's waterfall while controlling your own local SDR.

11.5 TLS pinning (optional — when OpenSSL is built in)

By default Kujhad rides on plain HTTP over your private overlay — the overlay IS the trust boundary. If your build links OpenSSL, you can wrap the listener in TLS. The controller pins the server's SHA-256 fingerprint — same model as SSH host keys.

On the publisher: SYS → Kujhad TLS → Generate self-signed cert → write down the fingerprint → read it to the controller out-of-band (voice, paper — NOT the same network you're pairing on) → toggle TLS enabled.

On the controller: KUJ → Add Peer → toggle TLS → paste the fingerprint into *Pinned fingerprint*. On first connect the cert is compared against the pinned value. Mismatch = abort with loud warning.

11.6 Python backend → Kujhad device

To have the Linux backend consume from a C++ Kujhad device, set FLEET_NODES in /etc/predator-rf/predator-rf.env:

```
FLEET_NODES=alpha@192.168.1.10:41947:<api_key>:hackrf,bravo@192.168.1.11:41947:<api_key2>:rtlsdr
```

Format per node: node_id@host:port:api_key:hardware_code. The backend polls each node: events at 1 Hz, GPS at 1 Hz, full state every 5 s, timing telemetry every 30 s.

> **Hardware mismatch warning:** If the device reports a different hardware code than what's in FLEET_NODES, the backend logs a warning but continues. Update the env file when you can.

11.7 Kujhad troubleshooting

Symptom	Most likely cause	Fix
Peer red / never green	Network unreachable, port blocked, or API key mismatch	Ping the host; telnet the port; verify the key character-for-character
Peer green but no events	Mission mode is Manual on the peer	Change peer to Scan or Classify
401 on every request	API key wrong	Re-copy the key
TLS handshake failure	Wrong fingerprint pinned, or peer cert regenerated	Re-pin the new fingerprint after verifying out-of-band

Hardware mismatch warning	FLEET_NODES has wrong hardware code	Update env; restart predator-rf. Backend trusts the device regardless.

---

12. Geolocation — TDOA, the error ellipse, what confidence actually means

12.1 What TDOA is

TDOA stands for Time-Difference-of-Arrival. When two or more GPS-synchronized sensor nodes hear the same transmission, the tiny difference in *when* each node received it (multiplied by the speed of light) gives a curved line where the transmitter must be. With three or more nodes those curves intersect at a position fix.

Think of it as triangulation using time rather than angle — two nodes gives you a search arc, three nodes gives you a point on the map.

12.2 What you need for a fix

Requirement	Why it matters
≥ 2 distinct sensor nodes	Two readings from the same receiver give no time-difference information
GPS lock on each participating node	Without a position you can't do the geometry at all
GPS lock age < 60 s on each node (GPS_MAX_AGE_S)	A stale GPS fix means the node may have moved — the math breaks silently
Each node's hearings within a 5-second window of each other	TDOA assumes one transmission; measurements too far apart in time can't be compared
A common time reference	GPSDO/PPS-disciplined hardware → high trust; system-clock-only → low trust but still produces a fix

Inclusive by policy: Any GPS-equipped node participates, even cheap RTL-SDRs without a GPSDO. Their timing trust is capped at 0.5 instead of 0.98, and the fix confidence is scaled accordingly. A rough fix from four cheap nodes is still useful — it gives you a search area instead of nothing.

12.3 The math (for briefing TOC)

- **2 distinct nodes:** falls back to the midpoint of their positions, fixed confidence = 0.3 (then scaled by timing trust).
- **3+ distinct nodes:** iterative least-squares solve, 50 iterations; geometric confidence = min(0.95, 0.5 + 0.1·N), then scaled by timing trust.
- **Timing trust per node:** GPSDO + PPS + |offset| < 10 ms → 0.98 / GPSDO + PPS → 0.90 / GPSDO no PPS → 0.75 / NTP + |offset| < 25 ms → 0.70 / NTP + |offset| < 100 ms → 0.55 / NTP worse → 0.40 / System clock only → 0.30 / Any with last sync > 5 min → minus 0.20.
- **Final fix confidence** = geometric_confidence × mean(timing_trust of all participants).

12.4 The error ellipse on the map

Every TDOA fix is rendered with a 1σ error ellipse, not just a dot. This is the difference between a 50-metre fix and a 5-kilometre search area — and both look like the same dot without it.

• Base radius scales as 50 m + (1 − confidence) × 4950 m — a high-confidence fix shrinks toward 50 m, a zero-confidence fix grows toward 5 km.

• Eccentricity comes from your node geometry: clustered nodes → near-circular blob; nodes in a line → long thin ellipse perpendicular to the baseline.

• Rotation follows the principal axis of the node cluster, so the ellipse physically matches your fleet's geometry.

12.5 What 'confidence' actually means in three places

Where you see it	What it actually measures
track.confidence on the HITS row	Detection confidence — how sure the system is this is a real, persistent emitter. Starts at 0.1, climbs with corroborating observations.
track.location_confidence (drives the map ellipse)	Geolocation confidence — TDOA fix quality only. Independent of detection confidence.
assessment.confidence on the threat assessment	Assessment confidence — the intelligence layer's confidence in its threat level call. Independent of both above.

Example: A track can have high detection confidence (0.9 — definitely real), low location confidence (0.2 — five-km search area), and medium assessment confidence (0.5 — probably elevated threat but not certain). All three surfaced separately.

12.6 What happens with no GPS-synced nodes (single-phone operator)

If you have only one phone, TDOA does not run — you need ≥ 2 GPS-synced sensors hearing the same emission within a 5-second window. Fallback behaviours:

Default (RSSI_PROXIMITY_ENABLED=false):

• Tracks have no estimated map position of their own.

• The map shows your phone's GPS dot and breadcrumb trail with hit timestamps. You do the triangulation in your head by walking the area and noticing where the signal gets stronger.

• The CoT beacon (§ 16), if armed, falls back to the detecting node's GPS — meaning TAK shows a marker at your phone's position, not the emitter's. This means 'I'm here and I heard something,' not 'the emitter is here.'

12.7 RSSI proximity fallback (single-node coarse geolocation)

This is NOT real geolocation. Treat the circle as a search area, not a confirmed position. Never report these coordinates to higher echelon as an emitter fix.

When RSSI_PROXIMITY_ENABLED=true, single-node tracks get a coarse position estimate centred on your phone's GPS. The circle radius is estimated from the signal's power via free-space path-loss model, multiplied by an uncertainty factor.

Good for: Walking-the-perimeter recon (circle visually contracts as you approach). Coarse bucketing: 'within 100 m or within 5 km?'

Not for: Reporting actual emitter coordinates. Urban canyons or heavy clutter — bump RSSI_RADIUS_UNCERTAINTY_FACTOR to 3 or 4.

TDOA always wins: As soon as a second GPS-synced node hears the same emitter, the location method flips to TDOA and the proximity circle is replaced by the proper ellipse.

---

13. Track lifecycle — NEW → TRACKING → STABLE → COASTING → LOST

State	Triggered by	What this means for you
NEW	First detection	One sighting, not yet corroborated. Don't act on it alone.
TRACKING	≥ 3 observations	The system believes this is real. Scoring is live. Time to pay attention.
STABLE	≥ 10 observations	Well-characterised. AutoTasker may auto-task nodes here. High confidence.
COASTING	Track aged out of last-seen window but within replay window	'Was here, hasn't been heard in a while, expected to return.' Still in the live picture.
LOST	Beyond the replay window	Archived. Removed from the live picture but stays in the mission ledger.

The state appears on the HITS row and gates how the system behaves. AutoTasker and CoT escalation are both more conservative on NEW tracks.

---

14. Trust model — node trust score, timing trust, sensitivity trust

Every sensor node carries a composite trust score between 0.05 and 0.98. This score multiplies the weight of that node's observations during fusion, so a flaky cheap node doesn't drag down a high-quality fix.

```
operational  = base_trust × uptime_fraction × (1 - false_positive_rate)
multi_node   = multi_node_agreement × 0.2
hardware     = freq_stability × 0.3
             + sensitivity    × 0.3
             + timing         × 0.2
             + 0.2 (constant)
trust_score  = (operational + multi_node) × hardware
               × (0.7 if thermal_throttling else 1.0)
```

Component	What raises it	What lowers it
base_trust	Operator manually marks node as trusted	Default 0.6
uptime_fraction	Node reachable continuously	Network drops, restarts
false_positive_rate	Few unconfirmed-by-other-nodes hits	High solo-hit rate
multi_node_agreement	Hearings corroborated by ≥ 1 other node	Solo observations only
freq_stability_trust	GPSDO-disciplined oscillator (low PPM)	Stock RTL-SDR tuner (50 PPM)
sensitivity_trust	Low noise figure (Airspy R2 at 2.5 dB)	High noise figure (HackRF at 10 dB)
timing_stability_trust	GPSDO + PPS, low offset	NTP only, large offset, stale sync
thermal_throttling	Cool device	Hot phone in sun → 0.7× multiplier

---

15. The intelligence layer — anomaly flags → threat level → recommended action

Threat level	When it fires	Recommended action
unknown	No flags AND confidence < 0.3	continue_monitoring
low	At least one flag, low severity	continue_monitoring
medium	One high-severity flag OR ≥ 2 medium flags	increase_dwell_time
high	Two high-severity flags OR (one high + confidence ≥ 0.5)	focus_all_nodes (auto-tasks every TDOA-capable node to this frequency)
critical	Any critical-severity flag	alert_operator_immediately — NEVER auto-actioned

Tracks at **high** or **critical** set escalate_to_atak = true, which is one of the two gates the CoT emitter checks (§ 16).

The frequency-band context labels each track with its regulatory band — Aviation, VHF Public Safety, Marine VHF, UHF Public Safety, ISM 433/915/2.4 GHz, GNSS. This flows into the assessment summary so the operator sees 'Emitter at 162.5500 MHz (Marine VHF)' instead of just a raw frequency.

---

16. ATAK / TAK CoT integration — two-key gate, manual approval queue

16.1 The two-key gate

Predator RF starts in RX-only posture. Two flags must both be true for a CoT packet to leave the system:

1. cot_enabled (env var COT_ENABLED=true, or the toggle in SYS → ATAK / CoT) — the operator-level kill switch.
2. The track's most recent assessment must have escalate_to_atak = true (set automatically for high or critical threat levels).

Even an automated critical assessment cannot bypass the operator's cot_enabled flag.

16.2 The manual approval queue (third key for field use)

In the field, set COT_REQUIRE_MANUAL_APPROVAL=true (or the *Require manual approval* toggle in SYS). With this on, escalate_to_atak no longer auto-fires. Each escalation queues at GET /api/v1/approvals and waits for the operator to explicitly approve before the packet goes out. Reject drops it; expire happens after 2 hours by default (COT_APPROVAL_EXPIRY_S).

> **Use this in the field.** This is the two-person-rule equivalent for a solo operator. A single false-positive can spam the TOC feed otherwise.

16.3 What goes on the wire

CoT 2.0 XML over UDP, defaulting to multicast 239.2.3.1:6969 (the TAK SA feed). For unicast to a TAK Server, override COT_DEST_HOST and COT_DEST_PORT.

CoT field	What's in it
type (geolocated emitter)	a-u-G (unknown ground unit) — when there's a TDOA fix
type (un-geolocated / LOB)	b-m-p-s-p-loc (point of interest) — when only a fallback or LOB-only location

type (sensor node SA beacon)	a-f-G-E-S (friendly ground electronic sensor) — the sensor node's own position beacon, so your nodes appear on the TAK map as sensor icons
point lat / lon	The TDOA fix, or the most-trustworthy node's GPS as fallback
point ce (circular error)	Scales 50 m (high confidence) → 5 km (zero confidence). TAK renders this as the circle around the marker.
point hae / le	9 999 999 (unknown altitude) — the platform does not claim altitude information
uid	COT_UID_PREFIX.emitter_id (dot separator)
contact callsign	COT_UID_PREFIX-emitter_id_first_8_chars (dash separator — the label visible on the ATAK map)
remarks	PREDATOR-RF | MHz | obs= | conf= |
stale	Default 5 min after emit (COT_STALE_S), then TAK fades the marker
__group name='Cyan' role='Team Member'	Renders as a friendly contact, not a hostile

Per-emitter rate limit: 5 seconds between beacons for the same emitter so a chatty source can't flood the TOC.

16.4 Test the path before you go live

The CoT export pull endpoint lets you verify the TAK pipeline works before the field:

```
curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/api/v1/cot/export
```

This returns all tracks that currently pass the escalation gate as CoT XML. Pipe it to a file and import via ATAK's CoT file import — if sensor and emitter markers appear you're wired correctly. No synthetic beacon is injected; what you see is what the system would actually send.

To test a single specific track:

```
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8000/api/v1/cot/export?emitter_id=<id>"
```

On mobile (ATAK plugin pull mode): When multicast UDP isn't available (carrier NAT, hotspot, school network), the ATAK plugin polls GET /api/v1/cot/export directly and feeds the XML into ATAK's local CoT pipeline. No extra config needed — it falls back to pull mode automatically.

---

17. AutoTasker — the action loop and its three brakes

When a **high** assessment recommends focus_all_nodes, AutoTasker re-tunes every TDOA-capable node to the track's primary frequency.

Critical assessments are never auto-actioned — the operator pushes the button.

Three brakes prevent runaway tasking:

1. **Per-node rate limit** — 30 s between tunes per node by default (AUTO_TASKER_MIN_INTERVAL_S). Prevents a chatty emitter from thrashing one node.
2. **Already-tuned check** — skips the tune if the node is already within ±2 kHz of the requested frequency.

3. **Global per-fleet budget** — at most 30 tunes/min across the entire fleet by default (AUTO_TASKER_GLOBAL_MAX_PER_MIN). Sized for ~6 nodes worth of churn. Prevents an assessment-loop bug from thrashing every node at 0200.

AutoTasker is OFF by default (AUTO_TASKER_ENABLED=false). Re-tune commands modify the SDR posture, so this is an explicit opt-in.

---

18. Operator overrides — friendly list, blacklist, manual location

Three classes of override, all persistent across restarts:

18.1 Friendly list

Mark an emitter_id as own-force or known-benign (your team's GMRS handhelds, the local police scanner, your own backhaul). Effect:

- Suppresses AutoTasker tunes for that emitter
- Suppresses CoT escalation
- Tags the track friendly in the UI (different icon, different colour)

Always recoverable — un-friend in the same UI.

18.2 Frequency blacklist

Add (start_hz, end_hz) ranges the SweepCoordinator must skip and the TrackManager must drop on ingest:

- A noisy commercial broadcaster you don't want clogging the HITS list
- A known interferer (your own LO leakage)
- An off-limits band you have a regulatory obligation NOT to log

18.3 Manual location override

Operator supplies a manual lat/lon for an emitter_id (confirmed via DF gear or visual). Wins over any TDOA estimate until cleared. Confidence is pinned to 0.95. The map ellipse shrinks accordingly.

All three overrides live in the mission database and rehydrate on restart. Every add/remove is stamped so AAR exports show exactly when an operator declared something friendly or moved it manually.

---

19. Mission lifecycle — start, run, end, export the AAR

19.1 Start

```
TOKEN=$(grep API_BEARER_TOKEN /etc/predator-rf/predator-rf.env | cut -d= -f2)
curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions \
  -d '{"name":"OVERWATCH-20260315","operator":"K9-Actual"}'
```

Or from the phone: MIS tab → Mission section → *Start mission* → enter name → Start.

Starting a new mission auto-ends any in-flight one — no need to remember to close the previous mission first.

19.2 Check active mission

```
curl -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/missions/active
```

19.3 End

```
curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions/end
```

19.4 Export the after-action package

```
curl -H "Authorization: Bearer $TOKEN" -OJ \
  localhost:8000/api/v1/missions/<mission_id>/export
```

Bundles a JSONL tarball: every event, track, assessment, approval (approved AND rejected), and override change. Time-stamped and self-contained.

---

20. Path 2: the Python backend (TOC workstation + RPi sensors)

> **Two separate programs:** predator-rf.service (Python, port 8000) is the intelligence backend — track fusion, TDOA, decision engine, CoT. predator-rfd (C++ web backend, port 5555) is the headless sensor daemon for a Linux node acting as a Kujhad Device. They are not interchangeable; they serve different roles.

20.1 Install (one-time)

On a Linux box (Debian / Ubuntu / Raspberry Pi OS):

```
sudo mkdir -p /opt/predator-rf /etc/predator-rf /var/lib/predator-rf/backups
sudo git clone <repo> /opt/predator-rf
cd /opt/predator-rf
sudo python3 -m venv venv
sudo venv/bin/pip install -r requirements.txt
sudo cp deploy/predator-rf.env.example /etc/predator-rf/predator-rf.env
sudoedit /etc/predator-rf/predator-rf.env       # see § 20.3
sudo cp deploy/predator-rf.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now predator-rf
```

20.2 Where things live

Path	What it is
/opt/predator-rf	Source checkout + Python venv
/etc/predator-rf/predator-rf.env	All env-var config
/var/lib/predator-rf/mission.db	SQLite mission ledger
/var/lib/predator-rf/backups/	Snapshots from deploy/backup_mission.sh
/etc/systemd/system/predator-rf.service	systemd unit
journalctl -u predator-rf -f	Live log tail command

Backend listens on :8000. For TLS, terminate at nginx / Caddy / Traefik in front — the backend stays plain HTTP intentionally.

20.3 The full env-var reference

Every knob, defaults shown. Edit /etc/predator-rf/predator-rf.env.

```
# API
API_HOST=0.0.0.0
API_PORT=8000
API_BEARER_TOKEN=           # empty = open (lab only); strongly recommended for any LAN deploy

# Fusion
```

```
TRACK_MAINTENANCE_S=10.0
TRACK_MERGE_S=30.0
MIN_CONFIDENCE=0.3

# Baseline learning
BASELINE_WINDOW_H=24.0
BASELINE_PRUNE_H=6.0

# Kujhad fleet (format: node_id@host:port:api_key:hardware_code)
# Port must match the Device's listen port (default 41947 for C++ nodes)
FLEET_NODES=alpha@192.168.1.10:41947:KEY:hackrf,bravo@192.168.1.11:41947:KEY:rtlsdr

# TDOA
TDOA_ENABLED=true
GPS_MAX_AGE_S=60.0
TIMING_POLL_INTERVAL_S=30.0

# Persistence
PERSISTENCE_ENABLED=true
DATA_DIR=/var/lib/predator-rf
MISSION_DB=mission.db
TRACK_REPLAY_WINDOW_H=24.0

# CoT (RX-only by default)
COT_ENABLED=false
COT_DEST_HOST=239.2.3.1
COT_DEST_PORT=6969
COT_UID_PREFIX=PREDATOR
COT_STALE_S=300.0
COT_MULTICAST_TTL=1
COT_REQUIRE_MANUAL_APPROVAL=false      # SET TRUE IN THE FIELD
COT_APPROVAL_EXPIRY_S=7200.0
COT_APPROVAL_MAX_PENDING=200

# AutoTasker (off by default)
AUTO_TASKER_ENABLED=false
AUTO_TASKER_MIN_INTERVAL_S=30.0
AUTO_TASKER_GLOBAL_MAX_PER_MIN=30

# CoC mode (TOC-of-TOCs)
COC_MODE_ENABLED=false
COC_UPSTREAM_URLS=           # CSV: http://stationA:8000,http://stationB:8000
COC_RECONNECT_DELAY_S=5.0
COC_DEDUP_INTERVAL_S=15.0
COC_DEDUP_FREQ_TOL_HZ=5000.0
COC_DEDUP_LOC_TOL_M=500.0

# Observability
LOG_LEVEL=INFO
LOG_FORMAT=text              # 'json' for Loki/Splunk/journald ingest
METRICS_ENABLED=true
SHUTDOWN_DRAIN_TIMEOUT_S=5.0
```

20.4 Day-of operations (Linux side)

```
sudo systemctl restart predator-rf           # apply env changes
sudo systemctl status predator-rf
journalctl -u predator-rf -f                 # tail logs
curl -s localhost:8000/healthz               # quick health check
```

```
curl -s localhost:8000/metrics               # Prometheus metrics
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/nodes    # fleet
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/tracks   # live tracks
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/approvals # pending
```

20.5 RPi sensor node

A Raspberry Pi running the Predator RF C++ build acts as a Kujhad Device. Drop it at a fixed point with:

- An SDR (RTL-SDR Blog v4 minimum, Airspy or HackRF preferred for sensitivity)
- A GPS HAT or USB GPS (Adafruit Ultimate GPS HAT, BU-353N5)
- Power (PoE injector or 12 V → USB-C PD)
- Networking (Ethernet preferred; ZeroTier / Tailscale installed for off-LAN reach)
- An antenna mounted as high as your mast allows

Configure its Kujhad listen address + port + API key. Add it to FLEET_NODES on the operator workstation. Done — the operator now sees its picture.

20.6 CoC mode (Center of Control — TOC of TOCs)

Set COC_MODE_ENABLED=true + COC_UPSTREAM_URLS=http://station-alpha:8000,http://station-bravo:8000 and this backend additionally consumes events from those upstream backends' SSE feeds. Every aggregated event is tagged with _upstream. CrossStationDedup coalesces tracks where frequency + location agree (±5 kHz, ±500 m, ±30 s co-occurrence).

20.7 RNS Layers (Reticulum transport for CoT)

The RNS layer is a parallel transport that sends the same CoT XML over Reticulum (RNS) at the same time as the regular TAK UDP path. RNS handles path selection across whatever interfaces you've brought up — LoRa, packet radio, I2P, mesh. This gets your CoT to TAK when regular IP is unavailable or untrusted.

The daemon ships in the Python backend (backend/rns/). Crypto stack: cbor2 envelopes, Argon2id (t=3, m=64 MiB, p=1) KDF, XChaCha20-Poly1305 IETF AEAD with version byte bound as AAD (downgrade-resistant).

Inbound CoT over RNS is auto-forwarded to a local TAK app over UDP. On Android the port defaults to 4242 so peer-relayed CoT shows on the device's TAK map without operator action.

20.7.1 Where to access it

Platform	How to open it	How it talks to the daemon
Linux	Kujhad panel → 'RNS Interfaces (Reticulum)' section	Unix socket (ControlServer), uid-checked, no network exposure
Android (Tier 2+ deployment)	Same Kujhad panel — identical layout to Linux	android.net.LocalSocket against the same Unix-socket path

> **No HTTP control plane.** HTTP routes in backend/api/routes/rns.py exist as scaffolding but are not mounted. The daemon control plane is local-only. To script it, use the Unix socket directly.

20.7.2 Interface types

Type	When to use it	Required fields

tcp_client	Reach a remote RNS node by dialing its TCP listener	target_host, target_port
tcp_server	Let other RNS nodes dial you on TCP	listen_port
udp	Cheap stateless UDP transport, LAN setups	listen_port
i2p	Anonymized transport over I2P	optional peers, connectable
auto_interface	Discover other Reticulum nodes on the same LAN automatically	group_id
rnode	LoRa via an RNode (most common long-range field setup)	port, frequency_hz, bandwidth_hz, txpower_dbm, spreadingfactor, codingrate
kiss_tnc	Generic packet-radio TNC running KISS	port, speed_baud
ax25_kiss	Amateur AX.25 packet radio (callsign required)	port, speed_baud, callsign, ssid, axint_port
pipe	Wrap an arbitrary external process as an RNS interface (advanced)	command

reliable_cot defaults: rnode → False (LoRa airtime is precious; unconfirmed send). Everything else → True.

20.7.3 Common fields (every interface)

Field	Default	Purpose
name	required	Human label, must be unique
type	required	One of the nine types above
enabled	true	Toggle without deleting
mode	full	RNS routing role. Use full unless you've read the RNS docs.
outgoing	true	Whether RNS may originate transmissions. RX-only nodes set false.
bitrate_hint_bps	unset	Hint for RNS scheduler. LoRa ≈ 1200, TCP ≈ 10■+
announce_interval_s	unset	How often this node announces itself
reliable_cot	type-default	Per-iface override of publish reliability mode
ifac_netname	unset	IFAC network name — scopes who can decode frames. Must pair with ifac_netkey.
ifac_netkey	unset	Pre-shared passphrase for IFAC. Treat as a secret — never logged in cleartext.
ifac_size	unset	IFAC hash truncation bytes (8..512). Leave unset for Reticulum's default.

20.7.3.1 IFAC — when to use it

IFAC is Reticulum's per-interface pre-shared-key gate. With both ifac_netname and ifac_netkey set, every frame is hashed with the netkey — nodes without the key can't decode link-layer framing at all.

Use when: sharing a link layer with other Reticulum users (open LoRa frequency, public TCP hub); multi-tenant LAN auto_interface; wanting defence-in-depth at link layer.

Don't use when: dedicated point-to-point link (peer allowlist already gates trust); operating under amateur radio rules that forbid encryption (IFAC may not be permitted on Part 97 frequencies — check local regs).

Both fields are required for IFAC to take effect. Setting only ifac_netname does nothing.

20.7.4–20.7.6 Adding interfaces, restart behavior, replication tokens

Adding an interface: Kujhad panel → RNS Interfaces → Add interface → pick type → fill required fields (form validates inline) → Save. Watch the live status row for online=true.

Restart behavior: Drains for drain_timeout_s (5 s default) → spawns new iface → waits for up. Forced teardown surfaces as last_error='forced'.

Replication tokens: A passphrase-encrypted bundle of the daemon config (and optionally its identity) for cloning RNS posture across nodes without retyping. Mint → copy the prf-rns-v1.* string → share out-of-band → Import on the other node → provide device-local placeholder values (serial ports, LAN IPs). Include_identity=true makes the importer adopt your node hash.

20.7.7 Peer allowlist

peer_allowlist (list of identity hashes) restricts which announces the bridge accepts. Leave empty for a closed deployment; populate for higher-trust ops.

20.7.10 Identity & state files (Linux)

Path	Purpose
/var/lib/predator-rf/rns/identity.prv	RNS identity private key (mode 0600) — back this up before any major change
/var/lib/predator-rf/rns/config.json	Daemon config — interfaces[], cot_bridge, peer_allowlist
/var/lib/predator-rf/rns/control.sock	Unix control socket (uid-checked)
/var/lib/predator-rf/rns/logs/	Rotating daemon logs

Back up identity.prv before any major change. Losing it means losing your node's RNS hash — every peer that allowlisted you needs to re-add the new hash.

20.7.12 Security posture

Threat	Mitigation
Remote attacker reaches daemon control plane	Not possible. Control plane is a Unix socket; HTTP routes not mounted.
Replication token sniffed	Argon2id + XChaCha20-Poly1305-IETF; downgrade-resistant. Useless without the passphrase.
Hostile peer floods the bridge	peer_allowlist + per-peer LRU dedup (4096 entries / peer) drops repeated CoT.
TX-class command injected	Dispatcher unconditionally rejects any tx-class command.

20.7.13 RNS troubleshooting

Symptom	Cause / fix
daemon=stub in status	rns Python package not installed. pip install -r backend/rns/requirements.txt and restart.

Iface online=false, last_error='...'	Read the error. Most common: serial port permissions (sudo usermod -aG dialout $USER), wrong port path, port already open.
LoRa peers can hear you but you never see them	Check announce_interval_s both sides. Set to 600 (10 min) for a stable mesh.
Iface online=true but peers can't see each other	IFAC mismatch — one side has IFAC, other doesn't. Clear IFAC on both or copy same values to both.
dropped_allowlist > 0	A peer is announcing but not in your allowlist. Add their hash or accept filtering.
Inbound CoT not in TAK on Android	Check RNS_ATAK_LOCAL_PORT (default 4242 on Android, opt-in elsewhere). ATAK must be listening on that port.
forced_close=true, timed_out=true	Iface hung on serial USB reset. Re-plug radio, click Restart again.

---

21. Hardware capability table (every supported SDR)

These drive the trust calculus. Pick your hardware knowing what trust score you'll get.

SDR	Freq range	Max sample	NF	MDS	TDOA	Timing unc.	Price
RTL-SDR Blog v4	25 MHz–1.7 GHz	3.2 MS/s	6.0 dB	–110 dBm	NO	1000 ns	$40
HackRF One	1 MHz–6 GHz	20 MS/s	10.0 dB	–100 dBm	YES	500 ns	$340
Airspy R2	24 MHz–1.7 GHz	20 MS/s	2.5 dB	–125 dBm	YES	50 ns	$170
LimeSDR-USB	100 kHz–3.8 GHz	61.4 MS/s	3.0 dB	–120 dBm	YES (PPS)	100 ns	$600
bladeRF 2.0	47 MHz–6 GHz	61.4 MS/s	4.0 dB	–118 dBm	YES	80 ns	$480
ADALM-PLUTO	325 MHz–3.8 GHz	61.4 MS/s	5.0 dB	–115 dBm	YES	200 ns	$200
SoapySDR generic	1 MHz–6 GHz	10 MS/s	8.0 dB	–105 dBm	NO	500 ns	varies

- **For TDOA:** Airspy or LimeSDR. Both have low timing uncertainty AND great sensitivity. LimeSDR's PPS output lets you discipline the LO from a GPSDO directly.
- **HackRF One:** The workhorse for HF coverage — the only SDR in the list that goes below 24 MHz. Acceptable for TDOA; not the best. At $340 it's the recommended upgrade from RTL-SDR.
- **RTL-SDR Blog v4:** The 'I'm here too' node. No TDOA timing path, so timing trust is capped at 0.5. Still useful as a 4th observer to pull a fix's confidence up.
- **PlutoSDR:** The budget TDOA node — half the price of an Airspy with TDOA capability.

---

22. Field-day checklist (print this)

Pre-departure (in shop / vehicle, with WAN)

- ■ Phone(s) charged + power bank packed
- ■ APK version verified — open app → SYS → check version against team's current build
- ■ SDR + antenna + USB-C OTG cable in the bag (one set per phone)
- ■ At least one baseline file for the area you're going to (record ahead if possible)
- ■ If using ATAK: TAK server reachable + credentials entered + a successful test beacon yesterday
- ■ If using Kujhad fleet: peers added, all on the same overlay, API keys match, TLS fingerprints pinned if TLS is on
- ■ If running the Linux backend: sudo apt update && sudo apt upgrade -y on workstation + each RPi
- ■ Mission DB backed up: deploy/backup_mission.sh to USB
- ■ System time disciplined: chronyc tracking shows Leap status: Normal
- ■ /etc/predator-rf/predator-rf.env reviewed; FLEET_NODES matches today's node serials
- ■ API_BEARER_TOKEN rotated for this mission: openssl rand -hex 32
- ■ CoT/TAK destination + UID prefix set ONLY if you intend to push to TOC
- ■ COT_REQUIRE_MANUAL_APPROVAL=true if COT_ENABLED=true
- ■ AUTO_TASKER_ENABLED matches your ROE — leave OFF unless authorized to re-tune nodes

On-site, before fleet power-on

- ■ Each sensor node placed; antennas oriented; GPS sky-view confirmed
- ■ Power budget sanity-checked (battery / vehicle alternator vs. node draw)
- ■ Network reachable: ping from operator workstation
- ■ Each node's clock disciplined (GPSDO PPS lock visible, or chronyc tracking green)

Bring-up sequence

1. Power on all sensor nodes; wait 60 s for GPS lock + Kujhad ready
2. Operator workstation: sudo systemctl start predator-rf
3. python deploy/preflight.py → must report GO
4. journalctl -u predator-rf -f → no ERROR lines in the first 30 s
5. curl http://localhost:8000/healthz → {"status":"ok"}
6. curl -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/nodes → every expected node listed, gps_synchronized=true on TDOA-capable ones
7. curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions -d '{"name":""}'
8. On the operator phone: SPEC → ■ Start. Verify waterfall + GPS green + KUJ green + CoT (if armed) green.

In-mission checks (every 30 minutes)

- ■ Glance at thermal indicator on each phone
- ■ Glance at GPS lock on each phone
- ■ curl /metrics → predator_events_total is climbing
- ■ curl /api/v1/android-pull?since_ns=0 → all nodes show gps_lock=true AND gps_age_s < 60

■ No CoT approvals stuck > 5 min in /api/v1/approvals (operator backlog)

■ Phone batteries > 20% — swap to power bank before 10%

End of mission

1. Phones: MIS → mode back to Manual; HITS → export hits to CSV; BASE → save fresh baseline if recorded
2. Phones: ■ Stop Listening → close app → unplug SDR
3. Workstation: POST /api/v1/missions/end
4. GET /api/v1/missions//export → save the AAR tarball
5. deploy/backup_mission.sh → USB

---

23. Troubleshooting (every symptom we've actually seen)

Symptom	Most likely cause	Fix
Waterfall is black after Start Listening	SDR not selected, or USB permission denied	Source dropdown → re-pick. Unplug + replug SDR → 'Always allow' on dialog.
App says 'device busy'	Another app or previous session holds the SDR	Force-stop in Settings → Apps; unplug + replug
Touch unresponsive / wrong widget fires	Old build	Update APK
Soft keyboard covers the input field	Old build — fixed by IME-inset clamp	Update APK
GPS never locks	Phone needs sky view; indoor/urban canyon = no fix	Step outside; wait 60 s
Letters don't appear when typing in popup	NativeActivity IME composing path	Switch to non-composing keyboard; numeric input always works
Kujhad peer red / disconnected	Network unreachable, port blocked, API keys differ	Ping the peer from a terminal app; verify keys character-for-character
Kujhad TLS handshake fails	Wrong fingerprint pinned, or cert was regenerated	Re-pin the new fingerprint (verify out-of-band first)
Kujhad peer green but no events	Peer is in Manual mission mode	Switch peer to Scan or Classify
ATAK sensor node markers missing (sensor SA)	COT_ENABLED=false, or sensor SA beacon not configured	Enable COT_ENABLED=true. Sensor nodes send their own a-f-G-E-S SA beacons; check sensor node's CoT config is active.
ATAK emitter marker never appears	Wrong TAK server IP/port, UID prefix filtering, or no track has TDOA fix + escalation	Check TAK server logs. Pull-test: curl -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/cot/export — if XML returns with events, backend is generating CoT correctly and the issue is on the TAK side.

ATAK markers stop coming	Approval queue on and operator hasn't approved	Check GET /api/v1/approvals; approve or disable manual approval
Phone hot, framerate drops	Thermal throttling	Get out of sun; reduce sample rate (SPEC → source settings); trust score drops 30% while throttled
App crashes on startup after install	APK sideloaded over a different signing key	Uninstall completely (Settings → Apps → Predator RF → Uninstall), reinstall
TDOA fix never appears on map	< 2 GPS-synced nodes hearing same emission within 5 s	Check /api/v1/nodes — at least 2 must show gps_synchronized=true and gps_age_s < 60
TDOA ellipse is huge (5 km)	Low-confidence fix — only 2 nodes, or all timing-trust-capped RTL-SDRs	Add more nodes; deploy at least one Airspy or HackRF with GPSDO
TDOA fix wildly off	One node's GPS is stale	GET /api/v1/nodes and check gps_age_s per node; offending one will be > 60 s
AutoTasker not retuning	AUTO_TASKER_ENABLED=false, OR global-budget brake firing	Check env, then /metrics for predator_autotasker_* counters
AutoTasker retuning the wrong nodes	DecisionEngine recommends only TDOA-capable nodes for high threats	This is by design; mark known-good non-TDOA nodes as friendly to suppress
Backend won't start: ERROR: address in use	Another process owns :8000	lsof -i :8000; kill or change API_PORT
Mission ledger growing huge	No prune configured	deploy/backup_mission.sh to archive; truncate per your retention policy
401 on every API call	API_BEARER_TOKEN set but no header sent	Add -H "Authorization: Bearer $TOKEN"

---

24. Bill of materials — pick your tier

Prices USD, June 2026, ballpark.

Tier 0 — Bare minimum (~$48)

- Android phone you already own — $0
- RTL-SDR Blog v4 — $40
- USB-C OTG adapter — $8
- Rubber-duck antenna (included with SDR) — $0

What you can do: Live spectrum, hits, baseline, scans up to 1.7 GHz. No HF, no fleet, no TDOA.

Tier 1 — Solo field operator (~$330–540)

- Tier 0 kit — $48
- Airspy Mini ($130) or HackRF One ($340)
- Diamond RH-77CA telescoping whip — $50
- External USB GPS (BU-353N5) — $35
- Powered USB-C OTG hub — $25

• 20,000 mAh USB-C PD power bank — $40

What you can do: All of Tier 0 plus HF coverage (HackRF), all-day battery, fast GPS, ATAK-ready.

Tier 2 — Solo TOC + one remote sensor (~$1,400)

• Tier 1 kit (HackRF flavor) — $540
• Raspberry Pi 5 8 GB + case + cooler + PSU — $130
• 256 GB A2 microSD — $25
• Second RTL-SDR Blog v4 (for the Pi) — $40
• GPS HAT (Adafruit Ultimate GPS HAT) — $45
• Linux laptop you already own OR add $500 for one
• Outdoor antenna mount + 25 ft LMR-400 + N-to-SMA pigtails — $80
• Diamond D130J discone — $130
• Pelican 1450 case — $130
• ZeroTier / Tailscale (free tier) — $0

What you can do: Drop-and-walk sensor node, distributed collection, mission ledger, after-action exports, ONE TDOA pair (phone + RPi).

Tier 3 — Small team / multi-node fleet ($5,000–$10,000+)

• Tier 2 kit — $1,400
• 3 more RPi sensor nodes (~$500 each) — $1,500
• At least one Airspy R2 + GPSDO for high-trust TDOA — $200
• Cellular hotspot + data plan — $300, OR mesh radio link (goTenna Pro X2 / RAJANT) — $500–$5,000
• Commercial directional antennas (Yagi for DF, dipoles per band) — $400
• 10–25 ft fiberglass tactical mast — $150
• Pelican / Storm cases per node — $400
• Operator laptop (rugged ThinkPad / Dell) — $500–$2,000

What you can do: Full-perimeter overwatch from one operator screen, real TDOA fixes with tight ellipses, ATAK to higher-echelon TOC, ledger across the fleet, real DF capability.

---

25. Glossary

Term	Meaning
AAR	After-Action Report — the mission ledger export tarball
AutoTasker	The action-loop module that re-tunes nodes based on threat assessments
Baseline	A recorded snapshot of 'normal' RF in an area, used for new-vs-known comparison
CE	Circular Error — the radius around a CoT marker representing 1σ position uncertainty
CoC	Center of Control — backend mode that aggregates from upstream backends (TOC of TOCs)
CoT	Cursor-on-Target — TAK's standard XML-over-UDP situational-awareness wire format
DecisionEngine	The module that turns tracks + anomalies into threat assessments

DSD-FME	The P25 P1+P2 decoder bridge
GPSDO	GPS-Disciplined Oscillator — locks the SDR's local oscillator for sub-ppm timing accuracy
HAE	Height Above Ellipsoid — CoT altitude field; Predator RF always sends 9999999 (unknown)
Hit	A persisted detection the operator (or system) chose to commit to memory
Kujhad	The Predator RF peer protocol (HTTP+JSON, X-Kujhad-Key auth, optional TLS pinning)
MDS	Minimum Detectable Signal — the noise-floor sensitivity in dBm. Lower (more negative) is better.
NF	Noise Figure — front-end noise contribution in dB. Lower is better.
OTG	USB On-The-Go — host-mode USB on the phone so it can power and command the SDR
PPS	Pulse-Per-Second — the 1 Hz timing pulse from a GPSDO that disciplines the SDR clock
ROE	Rules of Engagement
SAF	Storage Access Framework — Android's file picker; how the app does Import/Export
SDR	Software-Defined Radio — a radio receiver in software, typically a small USB dongle
TAK	Team Awareness Kit — the SA platform family (ATAK Android, WinTAK Windows, iTAK iOS)
TDOA	Time-Difference-of-Arrival — multi-receiver hyperbolic geolocation technique
TOC	Tactical Operations Center
Track	A fused, persistent identity for an emitter — unique by (frequency, modulation, detecting_node_set)
VFO	Variable Frequency Oscillator — in this app, one of two independent tuners

---

26. Quick reference card

Action	How to do it
START LISTENING	SPEC tab → source dropdown → ■ Start
DROP A MARKER	Tap a waterfall peak
NAME A MARKER	Tap marker → Assign Marker → tap 'Tap to edit'
RECORD BASELINE	BASE → +Add Range → ■ START RECORDING (5+ min) → Save
USE BASELINE	SYS → Baseline Comparison → load → enable
START AUTO SCAN	MIS → Mission Mode = Scan → +Add Band → SPEC → ■ Start
ADD KUJHAD PEER	KUJ → Add Peer → Name/Host/Port/Key → Add
PUBLISH ON KUJHAD	KUJ → Listen → port/name/key → toggle Listen
ENABLE ATAK	SYS → ATAK/CoT → server IP+port → Enable CoT (also: COT_REQUIRE_MANUAL_APPROVAL=true in field)
APPROVE A COT	GET /api/v1/approvals → POST /{id}/approve
TEST COT EXPORT	GET /api/v1/cot/export (returns live CoT XML for all escalated tracks)

START MISSION	MIS → Start mission → name (or POST /api/v1/missions)
END MISSION	MIS → End mission (or POST /api/v1/missions/end)
EXPORT AAR	GET /api/v1/missions//export

Status indicator	Meaning and action
THERMAL ORANGE	Reduce sample rate, get out of sun
GPS RED	Step outside, wait 60 s
KUJHAD RED	Check network, ping peer, verify API key
TDOA NO FIX	Need ≥ 2 GPS-synced nodes hearing same emission < 5 s apart
HUGE ELLIPSE	Low confidence — add more nodes / better timing hardware

Backend command	What it does
curl localhost:8000/healthz	Backend health check
curl -H 'Authorization: Bearer $TOKEN' .../api/v1/nodes	Fleet status
curl -H 'Authorization: Bearer $TOKEN' .../api/v1/tracks	Live tracks
curl -H 'Authorization: Bearer $TOKEN' .../api/v1/approvals	Pending CoT approvals
journalctl -u predator-rf -f	Live logs
sudo systemctl restart predator-rf	Restart backend
deploy/backup_mission.sh	Backup mission database

This document is the contract with the operator. If something here is wrong or out of date, fix it and commit. Rev 2 — June 2026.